

PARTS LIST

Semiconductors:

IC1	- AT89C51 microcontroller
IC2	- MAX232 driver
IC3	- 7805, 5V regulator
BR1	- 1A bridge rectifier
LED1-LED8	- 3mm light-emitting diode
LED9	- 5mm light-emitting diode

Resistors (all $\frac{1}{4}$ Watt, 45% carbon):

R1-R8	- 1-kilo-ohm
R9	- 10-kilo-ohm
R10	- 470-ohm
RNW1	- 10-kilo-ohm resistor network

Capacitors:

C1, C2	- 22pF ceramic
C3	- 10 μ F, 16V electrolytic
C4	- 1000 μ F, 25V electrolytic
C5	- 0.1 μ F ceramic
C6-C9	- 1 μ F, 16V electrolytic

Miscellaneous:

X1	- 220V AC primary to 9V, 250mA secondary transformer
X _{CRY}	- 11.0592MHz crystal
S1-S8	- 8-way DIP switch
S9	- Tactile switch
	- 9-pin D-type COM port connector

DC using a 7805 voltage regulator. To derive the power supply, the 220V, 50Hz AC mains is stepped down by transformer X1 to deliver a secondary output of 9V, 250 mA. The transformer output is rectified by bridge rectifier BR1, filtered by capacitor C1 and regulated by IC 7805 (IC3). Capacitor C2 bypasses the ripples from the regulated supply. LED1 acts as the power-'on' indicator. Resistor R1 limits the current through LED1.

Software

Java is a platform-independent language. So this software can be used in any operating system with the corresponding Java Runtime installed. The project contains Java source codes, Java classes, SIJJ_setup file and firmware.

We have used JCreator for developing the software. To use JCreator, you need JDK 1.6 or higher version installed in your system. The serial port is accessed by the Java program using third-party packages such as 'rxtxSerial.dll' and 'gnu.io'. These files are downloaded from the Internet.

All Java source files are in 'src' folder. Compile these using a Java editor such as JCreator v4.5 Pro. Compile Java files and generate a jar file in the folder.

On successful compilation of Java



Fig. 3: Tray icon



Fig. 4: Screenshot of Java program output

program, ensure that the following files are in 'Classes' folder:

1. 'Gnu' folder with 52 files
2. 'Pic' folder having two sub-folders with three files in the first folder and seven files in the second folder
3. 'SIJJ' folder with nine files
4. rxtxSerial.dll file of 75.9kB size
5. Manifest file of 71 bytes containing the text: "Main-Class: SIJJ.classes.SIJJ. SplashScreen-Image: pic/EG_Icon/EG.png"

Ensure that in the Manifest file there is a hard return (blank) at the end. That is, there should be three lines present with the last line blank.

The software for this project includes nine class files and a package for controlling the serial port. The main class file is SIJJ.class. This file creates the user-interface window for controlling the serial port. The main Java file, SIJJ.java, is listed at the end of this article for reference.

The MenuBarCreator.java file creates the menu bar and adds it to the window screen. The TrayCreator.java file creates an icon in the tray so that the program stays in the tray (refer Fig. 3) until you exit it. Clicking 'Close' button will not exit the program unless you click 'Exit' button in File menu.

The SendData.java file includes the code for controlling the serial port. This file uses the package 'gnu.io.~' that includes the class files for control-

ling the serial port. Here the code at the end of the SendData.java file is the standard code for initialising the serial port.

The getPortIds() method returns the available ports. The SimpleWrite() method sends the data to the hardware via the corresponding serial port. The 'public void recValue(int temprecval)' method is called whenever serial data is received by the corresponding port. When data is received, the work

to be done should be included in this function.

Third-party packages are included in 'Classes' folder for easy compilation.

It is necessary that the rxtxSerial.dll file is present in the same directory as 'Serial Interfacing In Java.jar' file. Or else, the .dll file should be put in 'java/bin' directory in Microsoft Windows. For working in Linux or Macintosh, you need other files, which are not described here.

SIJJ_setup file. This is the installer file for Windows operating system. You can run this file and choose a directory for installation. On successful installation, 'Serial Port Interfacing' folder is created, which includes files 'Serial Interfacing In Java.jar' and 'rxtxSerial.dll'. If you are using Windows 7 OS, some folders such as 'Program Files' need additional write permission. So install the file in another location such as the desktop. Ensure that you get a successful installation message.

'Serial Interfacing In Java.jar' is the final executable file. With JRE 1.6+ you can run jar file on Windows. The rxtxSerial.dll file is needed along with the executable file.

The Java software allows one to set the baud rate, data bit, etc. The screenshot of the program output is shown in Fig. 4.